



SWEN 221

Marco Servetto CO258 www.youtube.com/MarcoServetto

SWEN 221

Welcome.

SWEN221 is going to be a very hard course.

Very programming intensive.

In this course we will see true OO programming style.

We will see how OO is all about dynamic dispatch.

What is the right size for code on the slides?

```
List<String> log= new ArrayList<>(); //14pt
```

```
List<String> log= new ArrayList<>(); //15pt
```

```
List<String> log= new ArrayList<>(); //16pt
```

```
List<String> log= new ArrayList<>(); //18pt
```

```
List<String> log= new ArrayList<>(); //20pt
```

```
List<String> log= new ArrayList<>(); //22pt
```

```
List<String> log= new ArrayList<>(); //24pt
```

```
List<String> log= new ArrayList<>(); //28pt
```

```
List<String> log= new ArrayList<>(); //32pt
```

Lectures

- Tuesday 10:00 - 10:50 – MT228 Student Union, Kelburn
- Thursday 10:00 - 10:50 – MT228 Student Union, Kelburn
- Friday 10:00 - 10:50 – MT228 Student Union, Kelburn

We will use ALL the slots. I will show you A LOT of actual code.

- Marco Office hours in person: 11-13 Tuesday and Friday
 - Come to see me! If you have any doubts or questions, or you just want to learn more. My office hours are reported by many students as a great learning experience.

Assessment

- WAT 8% Due across the course
- Labs 11*2% Weeks 2, 3, 4,... 12

NO Slip Days

- Now: WAT DEMO
<https://ecs.wgtn.ac.nz/cgi-bin/swen221-questions>



4 term test, but no final exam

- Term Test 1 14% Mon23th March 5pm-8pm
- Term Test 2 14% Mon 04th May 5pm-8pm
- Term Test 3 14% Mon 25th May 5pm-8pm
- Term Test 4 28% In the examination period (40% score mandatory requirement)

No final exam.



Assessment Recap

- WAT 8% Due across the course
- Labs 11*2% Week 2,3,..
- Term Test 1 14% Mon23th March 5pm-8pm
- Term Test 2 14% Mon 04th May 5pm-8pm
- Term Test 3 14% Mon 25th May 5pm-8pm
- Term Test 4 28% In the examination period (40% mandatory requirement)



Final course mark

- Two options:
 - **Normal:** all the units of assessment will contribute to the final mark according to their percentage
If this marking strategy gives you a mark of 77% or more, this marking strategy is applied.
 - **Recovery:** we will ignore your performance on term test 1, 2 or 3; where you got the worst result. May be you just had a bad day.
The other two term tests will be scaled up to 21% so that the sum of them is still 42% of the final mark. The final course mark is then capped to 77%
- “**What if I'm sick on one of the term tests 1, 2 or 3?**”
 - We will consider on a case by case basis. In general if you present appropriate documentation we will use the '**Recovery**' option but not apply the 77% cap.



Marking expectations:

You may be under the impression that a student **barely passing** would be taking about 50% in all the assessment unites.

This is not the case in most second year courses.

Take at home assessments: (WAT+Labs) (30% tot marks)

A barely passing student would get more than 85% on those, if they put the effort in.

Term tests: 70% tot marks

A barely passing student would get about 35% on those.

$$(0.85 * 0.3) + (0.35 * 0.7) = 0.5 = C-$$

Labs are not easier then Tests, but Tests are conducted under exam conditions, presenting a significantly higher level of challenge, given the limited resources available.

Term tests

- Rules: just you and a browser running the WAT.
- As many WAT tabs as you want
- Hand out with some white paper and some help on Java provided
- **No access to other resources.**
- Term tests are run under exam conditions
- Bring the student Id, we will check you in as in exams.

Labs

- Week 1: no lab
- Week 2: The lab of week 2 is called `pre term test`. Each lab is worth 2%
- Week 3...Week 12: 10 more labs, 2% each. Most labs will have 50% automatically marked and 50% assessed by the tutors.
- Labs: (7 different time options!!)

1 Mon 11-13 CO243

2 Mon 13-15 CO243/CO242 (CO242 is overflow)

3 Mon 15-17 CO243/CO242 (CO242 is overflow)

4 Tue 11-13 CO243

5 Tue 14-16 CO243

6 Thu 11-13 CO243

7 Thu 13-15 CO243

<https://www.wgtn.ac.nz/students/study/timetables>
(to be activated 4pm on Thursday)

Term tests

- Rooms MC101/MC103 17:00–19:00
- Labs CO219/CO238/CO242 17:30–19:30

Your choice: Labs or Lecture room?

- Labs: you must use the labs computer, you should train yourself to use them well and fast.
- Lecture room: you must use your own laptop. There are electric plugs in the room.
- Do you prefer Lab or Lecture room?
- Book your space in My Allocator!
<https://www.wgtn.ac.nz/students/study/timetables>



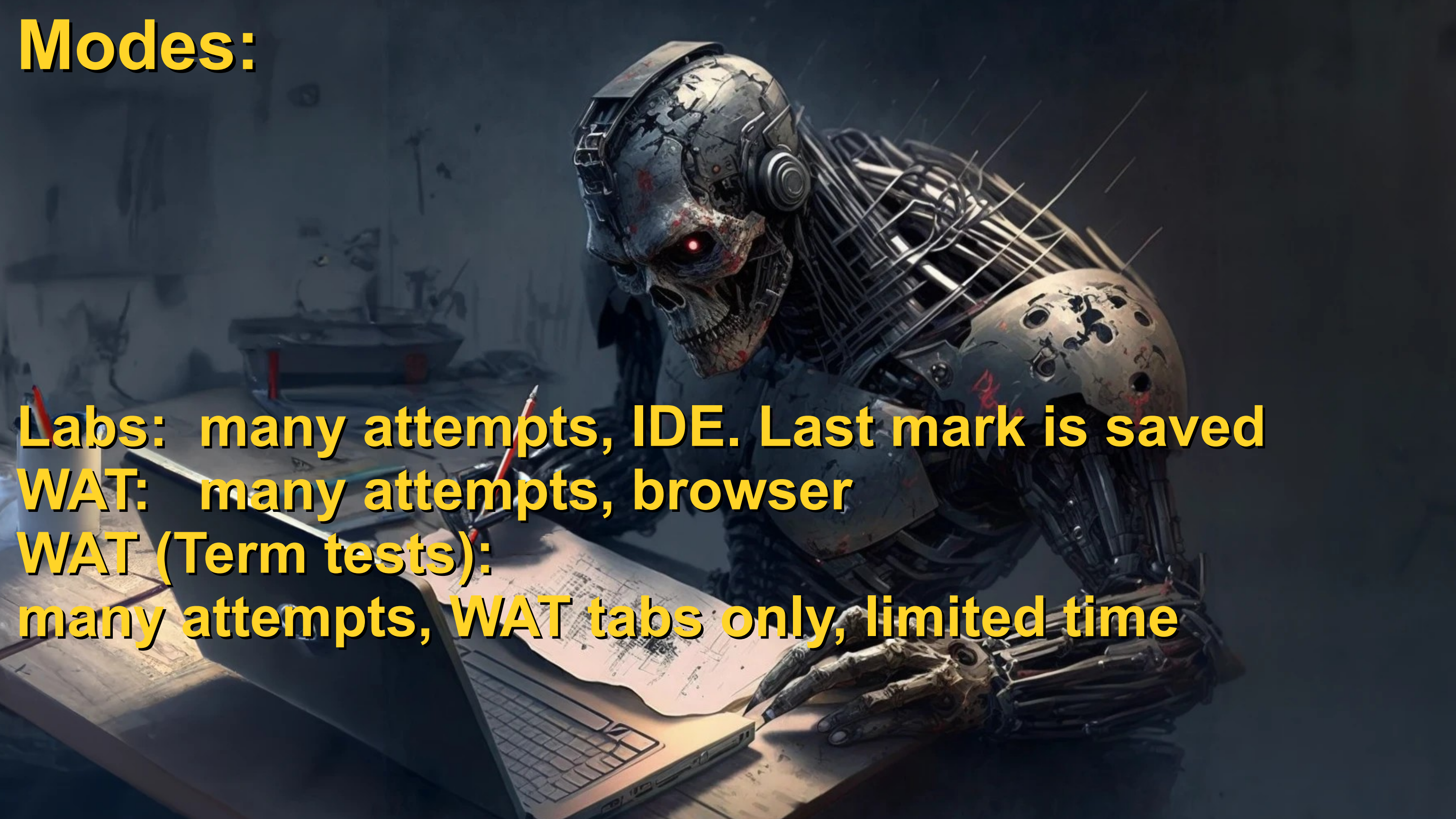
**The unstoppable
automatic
marking
system.**

**It never sleeps, It is never tired,
It has perfect attention.
Zero tolerance for mistakes.**



Modes:

Labs: many attempts, IDE. Last mark is saved
WAT: many attempts, browser
WAT (Term tests):
many attempts, WAT tabs only, limited time



Learning

- In the first year the learning was more guided:
 - The assignments and the labs were reinforcing all the content from the lectures needed to score well in the assessments.
- Now, at second year, learning is a little less guided:
 - **The lectures are the most fundamental place where the material is introduced.** The assignments and labs may reinforce some of the material, but large amounts of material introduced in the lectures will be assessed in the term tests directly.
 - You have to do independent exercises to get ready for the term tests.
- Going forward, the third and fourth year learning will have to be less and less guided since we need to teach you to learn independently.

Example from the past:

- Some of the easier questions in term test 1 required to use java 'record'.
- Records were presented in lecture 2 and used in lectures 3,4,...
- But that year no labs due before term test 1 used records explicitly.
- Many students that had only engaged with the labs, but not with the lectures could not pass those easy questions.

Autonomously apply language features

- Consider those two questions:
 - (A) “Using a `for' statement, sum all the elements of a list”
 - (B) “Sum all the elements of a list”
- (A) is telling you HOW to solve a problem, while (B) is asking you to have the independence needed to discover how to solve the problem on your own.
- Questions like (B) ask you to discover how to solve a problem instead of just checking that you can solve a problem **if** we tell you what tools to use.
- It is a fundamental part of making you into an independent programmer.
- Questions requiring you to **independently discover how** to solve them are not deceiving questions, but questions doing their work well.

How do I learn to independently discover how to answer questions?

- By coming to the lectures, where we will show you many, many examples of code and many examples of questions and solutions.
- We will show you the language features and how they work together.
- Labs will only use a few selected language features.
- Term tests will assess the full content of the lectures.
- **The lectures are the main source of learning for SWEN221**

A **bad** way to approach SWEN221

- Ignore the lectures at first, focus on the WAT and labs.
- When a WAT/lab refers to a new concept, google it and try to learn it independently, only if the googling fails then skim the lecture slides until you think you find where the concept is introduced.
- Try to pattern-match the data you have found to the problem at hand, without any attempt to understand or generalize the concepts.
- If this is the way you approach SWEN221, **you will fail all the term tests.**

Also, just reading the slides is not enough

- Here I'm putting down all my words in writing, so that if you are a student choosing to “*just read the slides*”, you will read that “*it is not sufficient to just read the slides*”.
- Either show up in person or (if desperate) watch the recordings.
- Plenty of crucial considerations are going to be expressed by commenting the slides via voice.

Workload estimation

SWEN221 is a regular 150 hours course.
You are paying for a 150 hours experience.

- 35 hours lectures
- 22 hours labs
- 8 for the 4 term tests
- 25 WAT at home
- Total: 90 hours of guided learning
- 60 hours left for independent learning / preparation:
about 15 hours of personal exercise/course review for each test
(note some labs/lecture serve as guided term test preparation too)

Early Deadlines:

- First group of WAT (week2)
- Labs Week 2,3,4,5...
- Term Test 1 (week 5)

Panic: Will I pass this course?

- It is a hard course.
- Assuming that you have spent 150 hours on each and every of the other programming intensive courses before this one...
- And that you are going to spend 150 hours on this course too..
- And that you **show up to all the lectures**, stay focus, and submit your work in time for all the labs, ...
- Then you will have a very high chance of passing.
- In person presence to lectures is going to help your learning. If you are **not in a special situation** → **you should attend in person**.
However, lectures are recorded. If you can not attend some lectures, you should send me an email every week with a summary of what you learned by watching the recording.

**If you can not attend some lectures,
you should send me an email every week
with a summary of what you learned
by watching the recording.**

Class REP!

- <http://www.vuwsa.org.nz/class-representatives/>



Java

Java is now an old language(1996)



But, recently.....
(after training in secret for 20 years)



Java: now a modern language!

Now with

- Records,
 - Interfaces with default methods
 - Lambdas
-
- We will use those features very intensively in this course!
 - We will use 'class' more rarely.

Hint!

When searching for Java examples, Java documentation or Java help

- Never search for “Java” you will get the old stuff.
- Search for “Java 25”, “Java 26” or “Java 2026” to get good stuff!



about coding

}

- General pattern: Start with a candidate result and improve it step by step while looking at the elements one at a time.

```
public static String longestString(List<String> ss){  
    String candidate= ???;  
    for (var s : ss){  
        ???  
    }  
    return candidate;  
}
```

- Start with a **candidate** result and improve it step by step while looking at the elements one at a time.
- Two questions:
 - What is the initial value for ‘candidate’
 - What do we do in the **for**?

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        ???  
    }  
    return candidate;  
}
```

- Start with a **candidate** result and improve it step by step while looking at the elements one at a time.
- Two questions:
 - What is the initial value for ‘candidate’
→ the first element of the list? An empty string? Null?
 - What do we do in the **for**?

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        ???  
    }  
    return candidate;  
}
```

- What do we do in the **for**? In the for we have two concepts: 'candidate' and 's'
- We need to improve 'candidate' by using 's'

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        if (???) { candidate = s; }  
    }  
    return candidate;  
}
```

- What do we do in the **for**? In the for we have two concepts: 'candidate' and 's'
- We need to improve 'candidate' by using 's'

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        if (s.length() > candidate.length()){ candidate = s; }  
    }  
    return candidate;  
}
```

- What do we do in the **for**? In the for we have two concepts: 'candidate' and 's'
- We need to improve 'candidate' by using 's'

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        if (s.length() > candidate.length()){ candidate = s; }  
    }  
    return candidate;  
}
```

??? *LongestString(List.of())* ??? *//What about empty lists?*

- This code nearly works, but.... not really!

```
public static String longestString(List<String> ss){  
    String candidate= ss.get(0);  
    for (var s : ss){  
        if (s.length() > candidate.length()){ candidate = s; }  
    }  
    return candidate;  
}
```

??? *LongestString(List.of())* ??? *//What about empty lists?*

- What happens when ss is empty?
- What should happen instead?

```
public static String longestString(List<String> ss){  
    if (ss.isEmpty()){ throw new IllegalArgumentException(); }  
    String candidate= ss.get(0);  
    for (var s : ss){  
        if (s.length() > candidate.length()){ candidate = s; }  
    }  
    return candidate;  
}
```

??? *LongestString(List.of())* ??? *//What about empty lists?*

- We need to consider error cases and handle them.
- This often happens by throwing our own errors.
- Remember: The empty list is a list. The empty list is a list. The empty list is a list.

```
public static String longestString(List<String> ss){  
    if (ss.isEmpty()){ throw new IllegalArgumentException(); }  
    String candidate= ss.get(0);  
    for (int i= 1; i < ss.size(); i += 1){  
        var s= ss.get(i);  
        if (s.length() > candidate.length()){ candidate = s; }  
    }  
    return candidate;  
}
```

- Alternative solution: by using indexes we can avoid checking what is longest between the 'first' and the 'first' element of the list.
I think the gain is very little and the result is ugly.

```
//using Integer.compare
public static String longestString(List<String> ss){
    return ss.stream()
        .max( (s1,s2)->Integer.compare(s1.length(),s2.length()) )
        .orElseThrow( IllegalArgumentException::new );
}
```

```
//using Integer.compare
public static String longestString(List<String> ss){
    return ss.stream()
        .max( (s1,s2)->Integer.compare(s1.length(),s2.length()) )
        .orElseThrow( IllegalArgumentException::new );
}
```

```
//even better using Comparator.comparingInt
public static String longestString(List<String> ss){
    return ss.stream()
        .max( Comparator.comparingInt(String::length) )
        .orElseThrow( IllegalArgumentException::new );
}
```

```
//using Integer.compare
public static String longestString(List<String> ss){
    return ss.stream()
        .max( (s1,s2)->Integer.compare(s1.length(),s2.length()) )
        .orElseThrow( IllegalArgumentException::new );
}
```

```
//even better using Comparator.comparingInt
public static String longestString(List<String> ss){
    return ss.stream()
        .max( Comparator.comparingInt(String::length) )
        .orElseThrow( IllegalArgumentException::new );
}
```

//We are going to reach this kind of programming!



- Better alternative solutions:
just code like a professional programmer instead of a first year student.
- This is using streams, which we have not seen yet, but we will see them soon.
- The streams will even force us to consider the empty list case.

See you Thursday!

Lecture 2:
Basic example programs
records and
the Post-it Model

